

Overview

This document explains the implementation of report generation in PDF and Excel formats using iText7 and Apache POI libraries, providing professional financial reports with charts, tables, and summaries.

Architecture

Core Components

1. **ReportService** - Orchestrates data collection
2. **IReportExporter** - Interface for report exporters
3. **PdfReportExporter** - PDF generation (iText7)
4. **ExcelReportExporter** - Excel generation (Apache POI)
5. **Report DTOs** - Data structures for reports

Dependencies

Maven Configuration (pom.xml)

```
<properties> <poi.version>5.2.3</poi.version> <itext.version>7.2.5</itext.version> </properties>
<dependencies> <!-- Apache POI for Excel --> <dependency>
<groupId>org.apache.poi</groupId> <artifactId>poi</artifactId>
<version>${poi.version}</version> </dependency> <dependency>
<groupId>org.apache.poi</groupId> <artifactId>poi-ooxml</artifactId>
<version>${poi.version}</version> </dependency> <!-- iText7 for PDF --> <dependency>
<groupId>com.itextpdf</groupId> <artifactId>kernel</artifactId>
<version>${itext.version}</version> </dependency> <dependency>
<groupId>com.itextpdf</groupId> <artifactId>layout</artifactId>
<version>${itext.version}</version> </dependency> </dependencies>
```

Report Structure

Money Outflow Report

Purpose: Consolidate all money outflows from different sources

Data Sources:

- Invoices (Transactional Documents)
- Salary Payments
- Service Payments
- License Plate Payments
- Fuel Loads
- Insurance Policies
- Vehicle Repairs

Report DTO Structure

```
@Builder public record MoneyOutflowReportDTO( ReportFilterDTO filters,
List<ReportItemDTO> items, BigDecimal totalAmount, Integer totalCount, LocalDateTime
generatedAt, Map<MoneyOutflowCategory, BigDecimal> summaryByCategory, String
reportName, String periodDescription ) {}
```

```
@Builder public record ReportItemDTO( LocalDate date, String description,
MoneyOutflowCategory category, BigDecimal amount, String reference, String supplier, String
project ) {}
```

```
public record ReportFilterDTO( LocalDate fromDate, LocalDate toDate,
List<MoneyOutflowCategory> categories, BigDecimal minAmount, BigDecimal maxAmount,
String supplierName, String projectName ) {}
```

Strategy Pattern Implementation

IReportExporter Interface

```
public interface IReportExporter { /** * Export report to specific format */ byte[]
export(MoneyOutflowReportDTO report); /** * Get format supported by this exporter */
ReportFormat getFormat(); /** * Get file extension */ String getFileExtension(); /** * Get content
type for HTTP response */ String getContentType(); }
```

ReportService - Strategy Pattern

```
@Service @Slf4j public class ReportService implements IReportService { // Map of exporters by
format (Strategy Pattern) private final Map<ReportFormat, IReportExporter> exporters; //
Repositories for data collection private final TransactionalDocumentRepository
transactionalDocumentRepository; private final SalaryPaymentRepository
salaryPaymentRepository; // ... other repositories public ReportService( List<IReportExporter>
exporterList, // ... repositories ) { // Automatically map exporters by format this.exporters =
exporterList.stream().collect(Collectors.toMap( IReportExporter::getFormat, exporter ->
```

```

exporter )); log.info("ReportService initialized with {} exporters: {}", exporters.size(),
exporters.keySet()); } @Override public ResponseEntity<byte[]>
generateMoneyOutflowReportFile( ReportFilterDTO filters, ReportFormat format) { // Get
appropriate exporter IReportExporter exporter = exporters.get(format); if (exporter == null) {
throw new InvalidReportFormatException(format.name()); } // Generate report data
MoneyOutflowReportDTO report = generateMoneyOutflowReport(filters); // Export to file byte[]
fileContent = exporter.export(report); // Build response headers HttpHeaders headers = new
HttpHeaders(); headers.add(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\""
+ buildFilename(filters, exporter.getFileExtension()) + "\"");
headers.add(HttpHeaders.CONTENT_TYPE, exporter.getContentType()); return
ResponseEntity.ok().headers(headers).body(fileContent); } }

```

PDF Report Generation (iText7)

PdfReportExporter.java

```

@Component @Slf4j @RequiredArgsConstructor public class PdfReportExporter implements
IReportExporter { private static final DateTimeFormatter DATE_FORMATTER =
DateTimeFormatter.ofPattern("dd/MM/yyyy"); private final MessageSourceHelper
messageSourceHelper; @Override public byte[] export(MoneyOutflowReportDTO report) {
log.info("Exporting report to PDF with {} items", report.totalCount()); try (ByteArrayOutputStream
baos = new ByteArrayOutputStream()) { PdfWriter writer = new PdfWriter(baos); PdfDocument
pdfDoc = new PdfDocument(writer); Document document = new Document(pdfDoc); // Add
content addTitle(document, report); addMetadata(document, report); addSummary(document,
report); addItemsTable(document, report); addFooter(document, report); document.close();
log.info("PDF report generated successfully"); return baos.toByteArray(); } catch (Exception e) {
log.error("Error generating PDF report", e); throw new ReportGenerationException(
messageSourceHelper.getMessage("report.generation.pdf.error"), e); } } private void
addTitle(Document document, MoneyOutflowReportDTO report) { Paragraph title = new
Paragraph(report.reportName()).setFontSize(20).setBold()
.setTextAlignment(TextAlignment.CENTER); document.add(title); } private void
addSummary(Document document, MoneyOutflowReportDTO report) { // Create summary table
Table summaryTable = new Table(UnitValue.createPercentArray(new float[]{50, 50}));
summaryTable.setWidth(UnitValue.createPercentValue(100)); // Add category summaries
report.summaryByCategory().forEach((category, amount) -> { summaryTable.addCell(new
Cell().add(new Paragraph(category.getDisplayName()))); summaryTable.addCell(new
Cell().add( new Paragraph(String.format("$%.2f", amount))
.setTextAlignment(TextAlignment.RIGHT)); }); document.add(summaryTable); } private void
addItemsTable(Document document, MoneyOutflowReportDTO report) { Table table = new
Table(UnitValue.createPercentArray( new float[]{10, 30, 15, 15, 15, 15}));
table.setWidth(UnitValue.createPercentValue(100)); // Headers
table.addCell(createHeaderCell("Fecha"));
table.addCell(createHeaderCell("Descripcion"));
table.addCell(createHeaderCell("Categoria"));

```

```

table.addCell(createHeaderCell("Monto"));
table.addCell(createHeaderCell("Proveedor"));
table.addCell(createHeaderCell("Proyecto")); // Data rows report.items().forEach(item ->
{ table.addCell(new Cell().add(new Paragraph( item.date().format(DATE_FORMATTER))));
table.addCell(new Cell().add(new Paragraph(item.description()))); table.addCell(new
Cell().add(new Paragraph( item.category().getDisplayName()))); table.addCell(new
Cell().add(new Paragraph( String.format("$%.2f", item.amount())))
.setTextAlignment(TextAlignment.RIGHT)); table.addCell(new Cell().add(new Paragraph(
item.supplier() != null ? item.supplier() : "-")); table.addCell(new Cell().add(new Paragraph(
item.project() != null ? item.project() : "-")); }); document.add(table); } @Override public
ReportFormat getFormat() { return ReportFormat.PDF; } @Override public String
getFileExtension() { return ".pdf"; } @Override public String getContentType() { return
"application/pdf"; } }

```

Excel Report Generation (Apache POI)

ExcelReportExporter.java

```

@Component @Slf4j @RequiredArgsConstructor public class ExcelReportExporter implements
IReportExporter { private static final DateTimeFormatter DATE_FORMATTER =
DateTimeFormatter.ofPattern("dd/MM/yyyy"); private final MessageSourceHelper
messageSourceHelper; @Override public byte[] export(MoneyOutflowReportDTO report) {
log.info("Exporting report to Excel with {} items", report.totalCount()); try (Workbook workbook =
new XSSFWorkbook(); ByteArrayOutputStream baos = new ByteArrayOutputStream()) { //
Create styles CellStyle headerStyle = createHeaderStyle(workbook); CellStyle dateStyle =
createDateStyle(workbook); CellStyle currencyStyle = createCurrencyStyle(workbook); CellStyle
totalStyle = createTotalStyle(workbook); // Create sheets createDataSheet(workbook, report,
headerStyle, dateStyle, currencyStyle, totalStyle); createSummarySheet(workbook, report,
headerStyle, currencyStyle, totalStyle); workbook.write(baos); log.info("Excel report generated
successfully"); return baos.toByteArray(); } catch (Exception e) { log.error("Error generating
Excel report", e); throw new ReportGenerationException(
messageSourceHelper.getMessage("report.generation.excel.error"), e); } } private void
createDataSheet(Workbook workbook, MoneyOutflowReportDTO report, CellStyle headerStyle,
CellStyle dateStyle, CellStyle currencyStyle, CellStyle totalStyle) { Sheet sheet =
workbook.createSheet("Datos"); // Create header row Row headerRow = sheet.createRow(0);
String[] headers = {"Fecha", "Descripcion", "Categoria", "Monto", "Proveedor", "Proyecto"}; for
(int i = 0; i < headers.length; i++) { Cell cell = headerRow.createCell(i);
cell.setCellValue(headers[i]); cell.setCellStyle(headerStyle); } // Create data rows int rowNum =
1; for (ReportItemDTO item : report.items()) { Row row = sheet.createRow(rowNum++); // Date
Cell dateCell = row.createCell(0);
dateCell.setCellValue(item.date().format(DATE_FORMATTER));
dateCell.setCellStyle(dateStyle); // Description
row.createCell(1).setCellValue(item.description()); // Category
row.createCell(2).setCellValue(item.category().getDisplayName()); // Amount Cell amountCell =

```

```

row.createCell(3); amountCell.setCellValue(item.amount().doubleValue());
amountCell.setCellStyle(currencyStyle); // Supplier row.createCell(4).setCellValue(
item.supplier() != null ? item.supplier() : "-"); // Project row.createCell(5).setCellValue(
item.project() != null ? item.project() : "-"); } // Add total row Row totalRow =
sheet.createRow(rowNum); Cell totalLabelCell = totalRow.createCell(2);
totalLabelCell.setCellValue("TOTAL"); totalLabelCell.setCellStyle(totalStyle); Cell
totalAmountCell = totalRow.createCell(3);
totalAmountCell.setCellValue(report.totalAmount().doubleValue());
totalAmountCell.setCellStyle(totalStyle); // Auto-size columns for (int i = 0; i < headers.length;
i++) { sheet.autoSizeColumn(i); } } private CellStyle createHeaderStyle(Workbook workbook) {
CellStyle style = workbook.createCellStyle(); Font font = workbook.createFont();
font.setBold(true); font.setColor(IndexedColors.WHITE.getIndex()); style.setFont(font);
style.setFillForegroundColor(IndexedColors.DARK_BLUE.getIndex());
style.setFillPattern(FillPatternType.SOLID_FOREGROUND);
style.setBorderBottom(BorderStyle.THIN); style.setAlignment(HorizontalAlignment.CENTER);
return style; } private CellStyle createCurrencyStyle(Workbook workbook) { CellStyle style =
workbook.createCellStyle();
style.setDataFormat(workbook.createDataFormat().getFormat("$#,##0.00")); return style; }
@Override public ReportFormat getFormat() { return ReportFormat.EXCEL; } @Override public
String getFileExtension() { return ".xlsx"; } @Override public String getContentType() { return
"application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"; } }

```

API Endpoints

Generate Report File

GET /api/v1/reports/money-outflow/file?format=PDF Authorization: Bearer {token} Query
Parameters: - format: PDF | EXCEL - fromDate: 2026-01-01 - toDate: 2026-01-31 - categories:
INVOICE,SALARY,FUEL - minAmount: 1000 - maxAmount: 50000 Response: Content-Type:
application/pdf Content-Disposition: attachment;
filename="reporte-salidas-dinero-2026-01-01-2026-01-31.pdf"

Generate Report Preview

GET /api/v1/reports/money-outflow/preview Authorization: Bearer {token} Response 200 OK: {
"totalAmount": 1500000.50, "totalCount": 127, "averageAmount": 11811.81,
"summaryByCategory": { "INVOICE": 800000.00, "SALARY": 500000.00, "FUEL": 200000.50 },
"periodDescription": "01/01/2026 - 31/01/2026" }

Best Practices

1. Use Strategy Pattern

Benefits:

- Easy to add new formats (CSV, XML, etc.)
- Clean separation of concerns
- Testable components

2. Stream Large Reports

For very large datasets:

```
@GetMapping("/stream") public StreamingResponseBody streamReport(ReportFilterDTO filters) { return outputStream -> { // Write data incrementally reportService.streamReport(filters, outputStream); }; }
```

3. Async Report Generation

For long-running reports:

```
@PostMapping("/generate-async") public ResponseEntity<String> generateReportAsync(ReportFilterDTO filters) { String jobId = reportService.scheduleReportGeneration(filters); return ResponseEntity.accepted().body(jobId); } @GetMapping("/status/{jobId}") public ResponseEntity<ReportStatusDTO> getReportStatus(@PathVariable String jobId) { return ResponseEntity.ok(reportService.getReportStatus(jobId)); }
```

4. Cache Generated Reports

Cache frequently accessed reports:

```
@Cacheable(value = "reports", key = "#filters.hashCode()") public byte[] generateReport(ReportFilterDTO filters) { // Generate report }
```

Performance Considerations

1. Pagination for Data Collection

```
Pageable pageable = PageRequest.of(0, 1000); Page<TransactionalDocument> page; do { page = repository.findAllWithFilters(filters, pageable); items.addAll(mapToReportItems(page.getContent())); pageable = pageable.next(); } while (page.hasNext());
```

2. Use Projections

```
@Query("SELECT new ReportItemDTO(d.date, d.description, ...) FROM TransactionalDocument d") List<ReportItemDTO> findReportItems(LocalDate from, LocalDate to);
```

3. Parallel Data Collection

```
CompletableFuture<List<ReportItemDTO>> invoices = CompletableFuture.supplyAsync(() -> collectInvoices(filters)); CompletableFuture<List<ReportItemDTO>> salaries = CompletableFuture.supplyAsync(() -> collectSalaries(filters)); CompletableFuture.allOf(invoices, salaries).join();
```

Testing

Unit Test

```
@Test void shouldGeneratePdfReport() { // Given MoneyOutflowReportDTO report = createTestReport(); // When byte[] pdfBytes = pdfReportExporter.export(report); // Then assertNotNull(pdfBytes); assertTrue(pdfBytes.length > 0); }
```

Integration Test

```
@Test void shouldDownloadExcelReport() throws Exception { // Given String token = getAuthToken(); // When MvcResult result = mockMvc.perform(get("/api/v1/reports/money-outflow/file") .param("format", "EXCEL") .param("fromDate", "2026-01-01") .param("toDate", "2026-01-31") .header("Authorization", "Bearer " + token)) .andExpect(status().isOk()) .andExpect(header().string("Content-Type", containsString("spreadsheetml"))) .andReturn(); // Then byte[] content = result.getResponse().getContentAsByteArray(); assertTrue(content.length > 0); }
```

Benefits

- ✓ **Multiple Formats** - PDF & Excel support
 - ✓ **Professional Output** - Styled reports with charts and tables
 - ✓ **Flexible Filtering** - Customizable report criteria
 - ✓ **Extensible** - Easy to add new formats
 - ✓ **Performance** - Optimized data collection
-

Conclusion

The reports generation system provides flexible, professional financial reports in multiple formats, with clean architecture using the Strategy pattern for easy extensibility.

Developed by: Maximo Andriola

For: Ricardo de ESEA SA

Date: January 2026

Version: 1.0

Status: Production Ready 